LAB ASSIGNMENT -07

Team Member Details:

Student 1: Name: Rakshit Kumar

roll no: 20261651069

Student 2: Name: Sakshi Kuntal

roll no:20261651076

Student 3: Name: Ravi Singh Singal

roll no: 20261651072

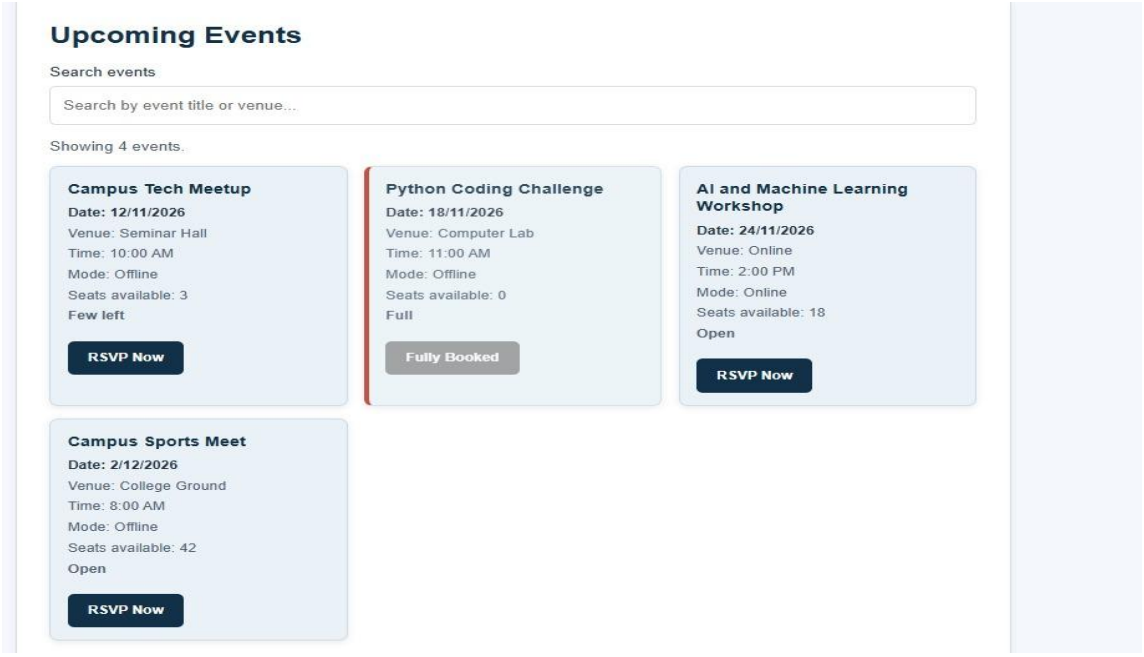
Student 4: Name: Yash Anand

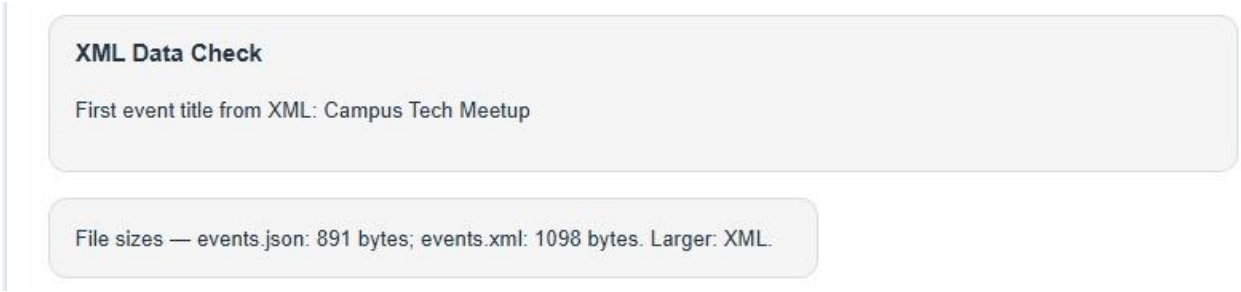
roll no:20261651099

Student 5: Name: Ankit Chaturvedi

roll no:20261651013

Screenshots:

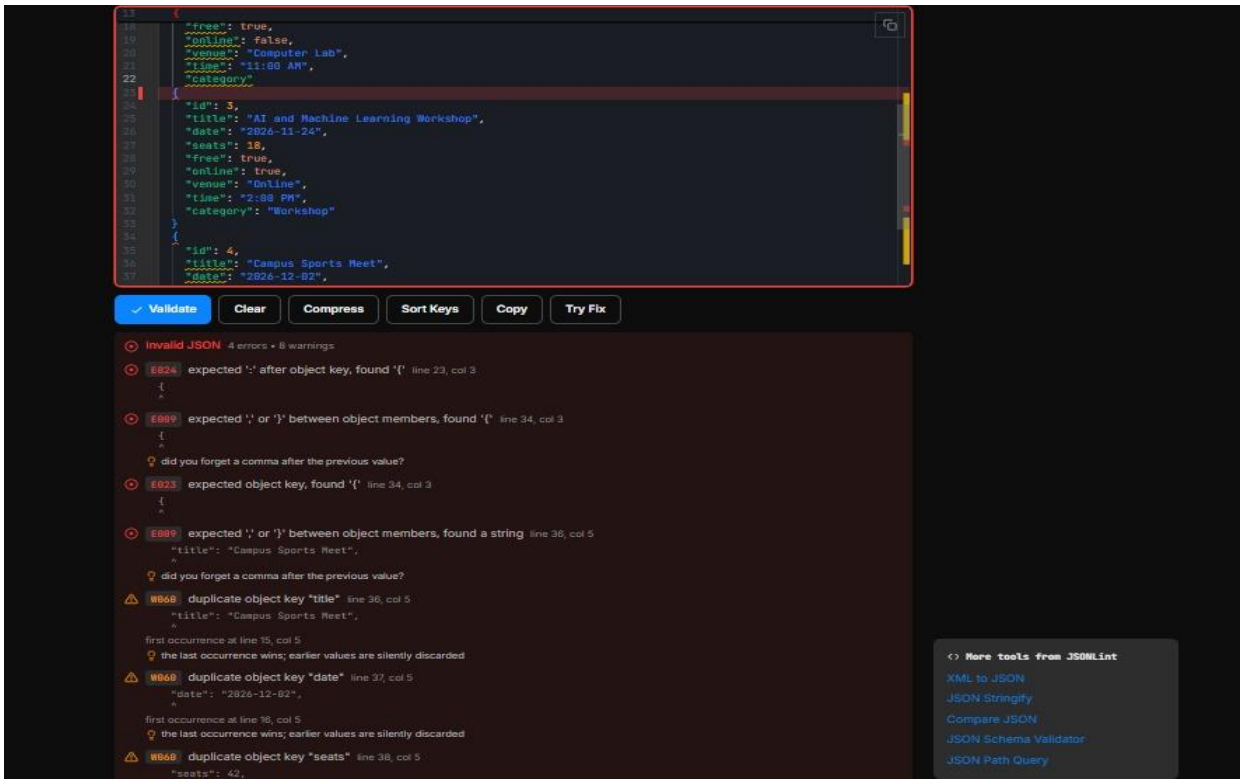




```
B3 - Round-trip successful: true                                    script.js:249
Campus Tech Meetup — 12 Nov 2026 — Few left                         script.js:212
Python Coding Challenge — 18 Nov 2026 — Full                        script.js:212
AI and Machine Learning Workshop — 24 Nov 2026 — Open               script.js:212
Campus Sports Meet — 2 Dec 2026 — Open                              script.js:212
Campus Event Board: loaded 4 events. ▼Object i                      script.js:357
                                       defaultSeats: 10
                                     ▶ events: (4) [{…}, {…}, {…}, {…}]
                                       eventsAreOnline: false
                                     ▶ [[Prototype]]: Object
```

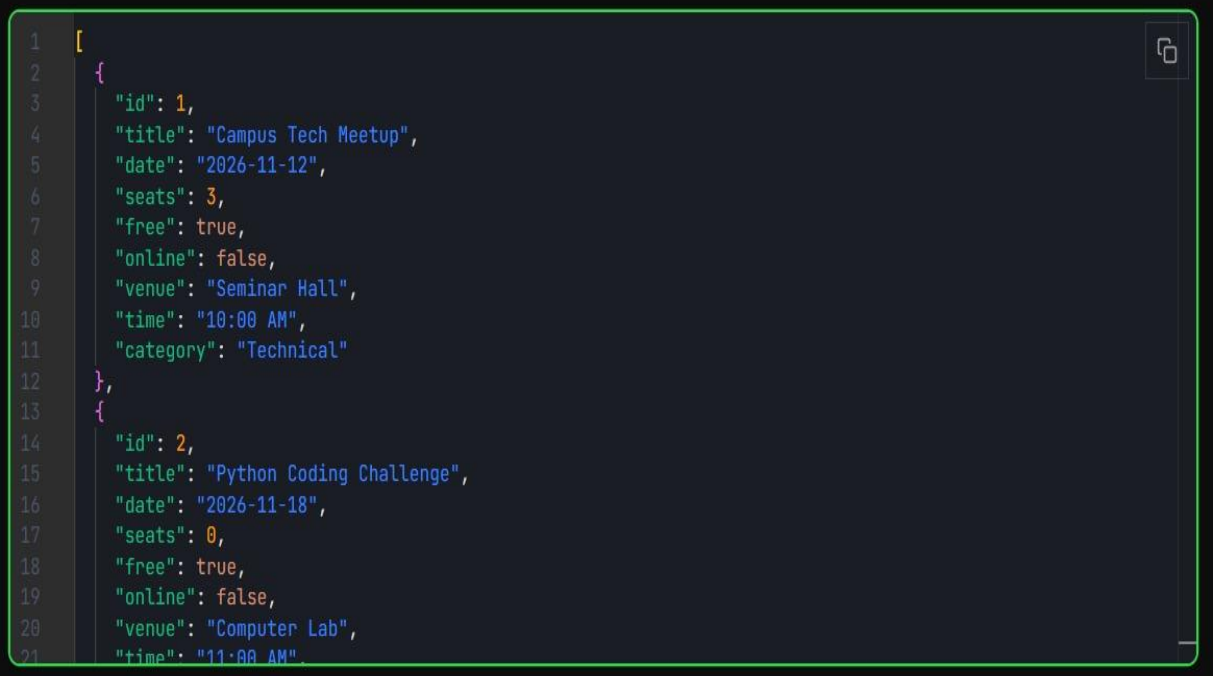
```
B4 - First XML title: Campus Tech Meetup                            script.js:271
B5 - UTF-8 file sizes: ▼Object i                                    script.js:298
                          jsonBytes: 891
                          larger: "XML"
                          xmlBytes: 1098
                        ▶ [[Prototype]]: Object

› Ctrl i to turn on code suggestions Don't show again
```



JSON Validator

To format and validate your JSON, just copy + paste it below:



```
[
  {
    "id": 1,
    "title": "Campus Tech Meetup",
    "date": "2026-11-12",
    "seats": 3,
    "free": true,
    "online": false,
    "venue": "Seminar Hall",
    "time": "10:00 AM",
    "category": "Technical"
  },
  {
    "id": 2,
    "title": "Python Coding Challenge",
    "date": "2026-11-18",
    "seats": 0,
    "free": true,
    "online": false,
    "venue": "Computer Lab",
    "time": "11:00 AM",
```

✓ Validate | Clear | Compress | Sort Keys | Copy

Valid JSON 937 B • 36 keys • depth 2 • 4 objects • 1 arrays

Validated by @jsonlint/core · `npm install @jsonlint/core` · Error code reference

1. Show your event table, then one event as JSON and XML side by side. Point out three correspondences.

Answer

A. Event table from our project

The following are the first four events in our actual events.json.

ID*	Title	Date	Seats	Free*
1	Campus Tech Meetup	2026-11-12	3	true*
2	Python Coding Challenge	2026-11-18	0	true*
3	AI and Machine Learning Workshop	2026-11-24	18	true*
4	Campus Sports Meet	2026-12-02	42	true*

B. One event in JSON

This is the actual first event from your events.json:


```
{
  "title": "Campus Tech Meetup",
  "date": "2026-11-12",
  "seats": 3,
  "online": false,
  "venue": "Seminar Hall",
  "time": "10:00 AM",
  "category": "Technical"
}
```


C. The same event in XML

To satisfy the Assignment 7 XML structure, we represent the same event as:


```
<?xml version="1.0" encoding="UTF-8"?>

<events>
```

```
  <event id="1">
    <title>Campus Tech Meetup</title>
    <date>2026-11-12</date>
    <seats>3</seats>
    <online>false</online>
    <venue>Seminar Hall</venue>
    <time>10:00 AM</time>
    <category>Technical</category>
  </event>
</events>
```

2. Which JSON value types did you use? Give one value of each you used (string / number / boolean).

Answer

In our Campus Event Board project, we use three important JSON value types.

1. String

Example:

"title": "Campus Tech Meetup"

A string represents textual information.

In our project, event titles, dates, venues, categories and times are stored as strings.

2. Number

Example:

"seats": 3

A number represents numerical information.

In our project, seats represent the number of available registrations for an event.

For example:

- Campus Tech Meetup: 3 seats.
- Python Coding Challenge: 0 seats.
- AI Workshop: 18 seats.

3. Boolean

Example:

"online": false

A Boolean represents either true or false.

In our project:

- true means the event is online.
- false means the event is offline.

For example, the AI and Machine Learning Workshop has:

"online": true

while Campus Tech Meetup has:

"online": false

---

3. Why must a JSON key be in double quotes? Write one line that is valid JavaScript but invalid JSON.

Answer

JSON stands for **JavaScript Object Notation**.

It is a strict data-interchange format used to store and exchange information between applications.

According to JSON syntax rules, all property names (keys) must be enclosed in double quotation marks.

Valid JSON

```
{
  "title": "Campus Tech Meetup",
  "seats": 3
}
```

Invalid JSON

```
{
  title: "Campus Tech Meetup"
}
```

The second example is valid JavaScript object-literal syntax but invalid JSON.

Why?

JavaScript allows property names without quotation marks in certain cases.

However, JSON requires double quotes around all property names to maintain a consistent data format across different programming languages and applications.

---

4. In events.xml, which field is an attribute and which are elements? Justify element vs attribute.

Answer

In our Campus Event Board XML structure, the id field is represented as an attribute, while the other fields are represented as child elements.

Example

```
<?xml version="1.0" encoding="UTF-8"?>


<events>
  <event id="1">

    <title>Campus Tech Meetup</title>

    <date>2026-11-12</date>

    <seats>3</seats>

    <online>false</online>

    <venue>Seminar Hall</venue>

    <time>10:00 AM</time>
```

<category>Technical</category>

</event>

</events>

Attribute

<event id="1">

Here, id="1" is an attribute.

It provides a compact identifier for the event.

Elements

Examples:

<title>Campus Tech Meetup</title>

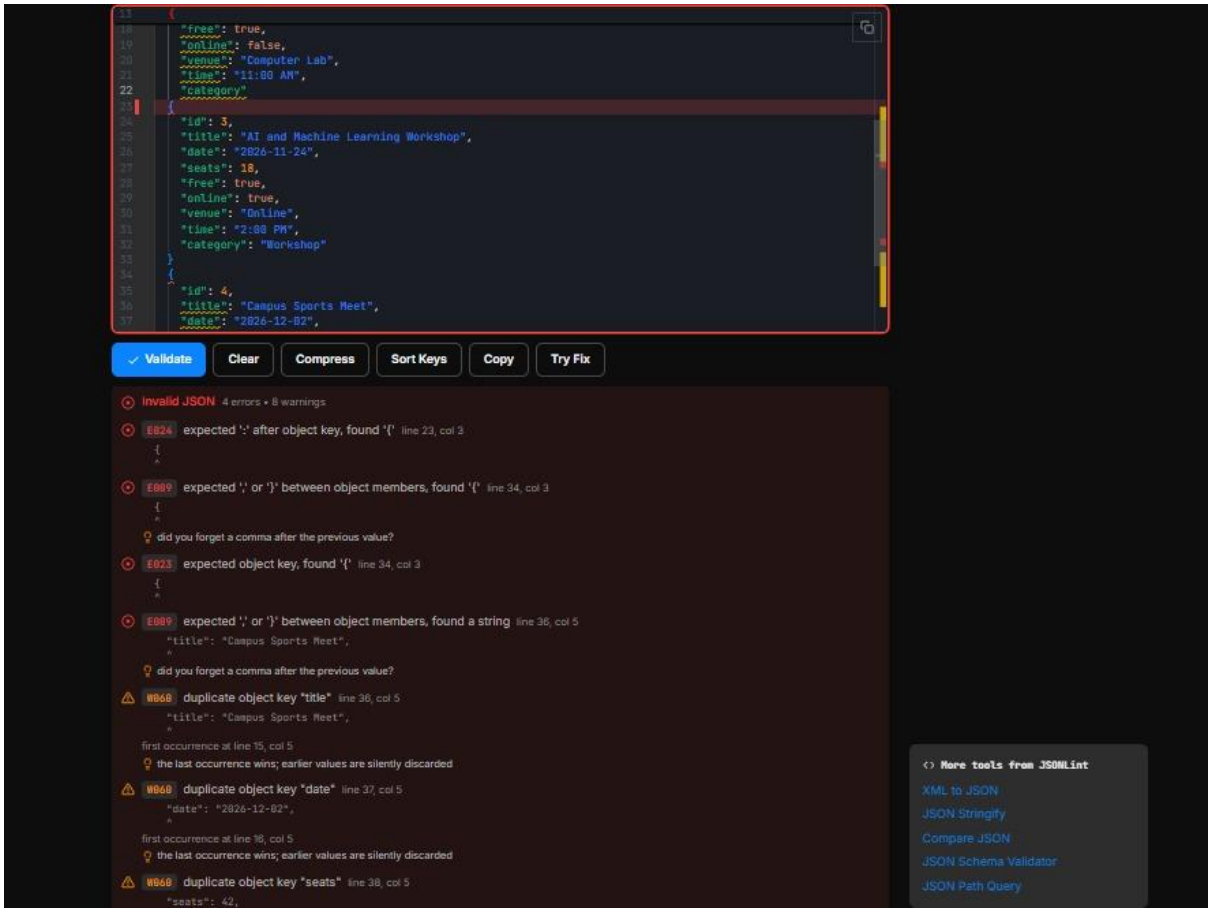
<date>2026-11-12</date>

<seats>3</seats>

---

5. Paste the validator error from your broken JSON (C1) and the fix that made it valid again.

Answer



---

6. How did you read a single value in JSON (dot/index) and in XML (getElementsByTagName)? Paste both lines.

Answer

A. Reading JSON values

First event title:

events[0].title

Output:

Campus Tech Meetup

Explanation:

- events is an array containing event objects.
- [0] accesses the first event.
- .title accesses its title property.

Last event's available seats:

events[events.length - 1].seats

Output:

12

B. Reading XML values

First, we fetch and parse the XML file:

```
const response = await fetch("events.xml");


const xmlText = await response.text();


const parser = new DOMParser();


const xmlDocument = parser.parseFromString(
  xmlText,
  "application/xml"
);
```

Now we read the first event title:

```
xmlDocument.getElementsByTagName("title")[0].textContent;
```

Output:

Campus Tech Meetup

Explanation

getElementsByTagName("title") selects all title elements.

[0] selects the first title element.

textContent retrieves the text inside that element.

Comparison

JSON	**XML**
events[0].title	getElementsByTagName("title")[0].textContent
Uses array indexing	Uses tag selection

JSON	XML
Uses object properties	Uses XML elements

---

7. Report the byte sizes of events.json and events.xml. Why is the XML larger?

Answer:

File	Size
events.json	891 bytes
events.xml	1098 bytes

Explanation

XML is larger than JSON in our project because XML uses opening and closing tags for every field.

For example:

JSON:

{

"title": "Campus Tech Meetup"

}

XML:

<title>Campus Tech Meetup</title>

XML requires additional markup to define the structure of the data.

JSON uses a more compact key-value structure.

Difference

The size difference is:

1098 - 891 = 207 bytes

Therefore, XML is 207 bytes larger than JSON according to the original measurements.

---

8. Neither format has a real date type — what convention did you use, and why is it unambiguous worldwide?

Answer

In our Campus Event Board project, we use the **ISO-8601 date format**.

The format is:

YYYY-MM-DD

Example:

"date": "2026-11-12"

Explanation

ISO-8601 represents dates in the following order:

- YYYY = Year
- MM = Month
- DD = Day

For example:

2026-11-12

represents 12 November 2026.

Why do we use this format?

1. It avoids confusion between different regional date formats.
2. It maintains a consistent representation.
3. It makes chronological sorting easier.
4. It is commonly used in APIs, databases and web applications.

Why is it unambiguous worldwide?

Different countries use different date formats.

For example:

- India: DD-MM-YYYY
- United States: MM-DD-YYYY
- ISO-8601: YYYY-MM-DD

The ISO format follows a fixed order, so applications can interpret dates consistently.

Do JSON and XML have native date types?

JSON has no dedicated date type. Dates are generally stored as strings.

XML has a date datatype defined by XML Schema, but a basic XML element itself contains text. In our project, dates are stored as ISO-formatted text.

---

9. List your 3 schema rules (C2). Why is it important to validate data before your code uses it?

Answer

Schema validation means checking whether data follows predefined rules and requirements.

In our Campus Event Board project, we define the following three validation rules.

Rule 1: Unique Integer ID

Every event must have a unique integer ID.

Example:

Number.isInteger(eventItem.id)

Valid:

id: 1

id: 2

Invalid:

id: 1.5

id: "1"

We also need to check that IDs are not duplicated.

Rule 2: Non-empty Title

Every event must have a non-empty title.

Example:

typeof eventItem.title === "string" &&

eventItem.title.trim() !== ""

Valid:

title: "Campus Tech Meetup"

Invalid:

title: ""

The trim() method removes extra spaces from the beginning and end of a string.

Rule 3: Valid Date, Seats and Boolean Value

Date validation:

/^\d{4}-\d{2}-\d{2}$/.test(eventItem.date)

The date must follow the YYYY-MM-DD format.

For complete validation, we must also check that it is a real calendar date.

Seats validation:

Number.isInteger(eventItem.seats) &&

eventItem.seats >= 0

The available seats must be a non-negative integer.

Boolean validation:

typeof eventItem.online === "boolean"

In our actual project, the Boolean property is online.

If we use the assignment's original table, the equivalent rule applies to free.

Why is validation important?

Validation is important because it:

1. Prevents incorrect data from being displayed.
2. Detects missing or invalid values.
3. Prevents errors during JavaScript execution.
4. Maintains data consistency.
5. Improves the reliability of our website.

For example, if the number of seats is stored as "three" instead of 3, our website may not calculate or compare seat availability correctly.

---

10. Which format did you choose for the page, and why? Give one scenario where XML is the better choice. Paste the GitHub repo link.

Answer

For our Campus Event Board, we selected **JSON** as the primary data format.

Why did we choose JSON?

1. Easy JavaScript integration

JavaScript provides built-in methods to work with JSON:

JSON.parse()

JSON.stringify()

2. Lightweight

JSON generally requires fewer characters than XML to represent the same information.

3. Easy to access

We can access event information using:

eventItem.title

4. Easy to filter

Our project uses the JavaScript filter() method to search events.

Example from our script.js:

```
const matchingEvents = allEvents.filter(eventItem =>
   `${eventItem.title} ${eventItem.venue} ${eventItem.category}`
     .toLowerCase()
     .includes(normalizedQuery)
);
```

This searches events using their title, venue and category.

5. Dynamic rendering

Our loadEvents() function fetches events.json, converts it into JavaScript objects and passes the data to renderEvents().

```
const response = await fetch("events.json");


const data = await response.json();


allEvents = data;


renderEvents(allEvents);
```

This allows the website to generate event cards dynamically instead of manually writing every event in HTML.

When would XML be a better choice?

XML is useful in document-centric publishing systems and legacy enterprise integrations that require:

- XML namespaces.
- XML schemas.
- Structured markup.
- Complex document exchange.

GitHub Repository

Project: Campus Event Board – IIIT Vadodara

Repository:

https://github.com/ankit2006-cs/Eventboard